Measuring Well-Being (SDQ) Platform: System Review and Migration Assessment

Date: 2026-10-18 Scope: the Laravel web application in this repository, the five documents retained under `docs/`, and the database backups under `database/Backup/`. Goal of the programme: migrate the platform to Azure with a modernised web application and mobile app, packaged as Docker containers.

1. What the system is

The platform lets UNICEF Lebanon and its child-protection partners run the Strengths and Difficulties Questionnaire (SDQ) as a pre-test and post-test around focused psychosocial support cycles. It reports on one indicator: the percentage of children whose psychosocial well-being improved between the two tests.

Two components exist:

- **Web dashboard** (this repository). Laravel 5.8 on PHP 7.1+, MySQL, Metronic 5 Bootstrap 4 admin theme, jQuery DataTables and amCharts. Hosted on GoDaddy shared hosting at measuringwellbeing.org (prod and staging folders).
- **Android app** (not in this repository). Native Java app, Android 5+, offline-first with a local SQLite store that syncs to the web API. Distributed as an APK from the website, version 10 at the time of the guidance note. The source was handed over as a zip file and must be located.

Three roles drive everything:

Role	Can do
Admin (UNICEF)	Manage partners, view all collectors, all applications, change-decrease follow-up, analytics, export, locations and statuses.
Partner (NGO)	Manage own collectors, see own applications, transfer an application between collectors, justify decreases, own analytics and export. Partners carry a prefix and a yearly application quota.
Collector (field staff)	Create pre-tests and post-tests via the Android app or the web form. Carries a prefix.

An **application** is one child's SDQ record. It moves through New, Pre Test, Pre Test Pending, Pre Test Done, Post Test, Post Test Pending, Done. Each test stores the 25 SDQ items, demographics, location (governorate, caza, area, GPS), consent, programme hours and dropout reason. Codes are generated client-side as `YEAR-PARTNERPREFIX-COLLECTORPREFIX-N`.

Scoring (from the scoring document and `app/Application.php`): each of the five scales is the average of its items times 5 plus 0.5, with NA items omitted. Total is the sum of emotional, conduct, hyperactivity and peer-problem scales. Pro-social is excluded. Only applications with a post-test count, and dropouts are excluded from change analysis. A lower post-test total than pre-test is labelled Increase in well-being.

2. Repository inventory

Area	Finding
Framework	Laravel 5.8 (end of life since 2020), PHP 7.1 constraint, `spatie/laravel-permission` 3.x, `yajra/laravel-datatables` 9.x.
Application code	14 web controllers and 5 API controllers, about 8,400 lines. `ReportController.php` alone is 2,920 lines and `ApplicationController.php` 1,314 lines, with a near duplicate in `Api/ApplicationController.php`.
Data model	12 tables: users, partners, collectors, application, application_value (one row per pre or post test, 70+ columns), gouvernate, caza, area, status, forms, plus Spatie permission tables.
Front end	Server-rendered Blade views (68 files), English and Arabic translations, Metronic theme assets. `public/assets` is 88 MB including a 10 MB demo folder.
Tests	Only the two Laravel example tests. No coverage of scoring, code generation or reports.
Vendored content	`vendor/` (55 MB) is committed. Compiled Blade views in `storage/framework/views` are committed. A GoDaddy `error_log` is committed. There is no `.gitignore` and no `.env.example`.
Database backups	Eight SQL dumps from December 2019 to January 2020, all from early development. The largest has one application row.

3. Key risks and findings

Ordered by how much they affect the migration.

1. **The migrations do not describe the production schema.** Code reads columns that no migration creates: `collectors.prefix`, the five `*_scale` columns and `na_count` on `application_value`, `reason_option` and `reason_details` on `application`, `users.api_token`, and the whole `application_log` table used by the reports page. The schema was evolved by hand on the server. The only authoritative schema is the live GoDaddy database, and it must be exported before anything else.
2. **The production data is not in the repository.** The dumps here hold test data from 2020. The description document reports 3,350+ real entries by May 2022, and more have accumulated since. A fresh full dump, plus the partner logos in `public/images/partner_logo`, is the real migration payload.
3. **Android app source is missing.** The technical doc refers to an `androidapp.zip` handed over separately. Without it the mobile client must be rebuilt, which the modernisation plan should assume anyway, but the current APK and its SQLite schema are needed to understand the offline sync contract.
4. **Business logic lives in the client.** Application codes and the increment counter are generated on the device, and the technical doc warns that using both the app and the web form causes code collisions. The new design should move code generation and conflict handling to the server with idempotent sync.
5. **Security weaknesses to fix in the rebuild, not patch.**

    - API tokens are 50 random characters stored in plaintext and passed as a `token` query parameter on every call, so they land in web server logs.
    - All API routes accept any HTTP verb (`Route::any`) and the API has no rate limiting beyond the default.
    - A public unauthenticated `/cache` route clears and rebuilds config cache.
    - Several web endpoints use `$request->all()` with no validation, and DataTables action buttons are built by string concatenation (possible stored cross-site scripting through partner or collector names).
    - Outgoing mail is hard-coded to send from a developer's Gmail test address.
    - The technical PDF contains plaintext FTP credentials. Rotate them.
6. **Performance problems visible in the log.** The committed `error_log` shows repeated memory exhaustion in the dashboard filter route, which loads all applications for every governorate into PHP. Export and list pages run one or two extra queries per row. Reports recompute every scale from raw item columns at request time.
7. **Scoring code does not match the scoring document in at least one path.** The web post-test save assigns a random score (`mt_rand(1, 20)`) before recalculating. It is overwritten later in the normal flow, but it shows the fragility of a 230-line hand-written scoring function with no tests. The rebuild needs the scoring rule implemented once, server-side, with unit tests against the scoring document.
8. **Licensing.** The UI is built on the commercial Metronic theme. The licence must be confirmed or the theme replaced in the new front end.

4. What is sound and should be preserved

- The domain model is simple and clear: partner, collector, application, two test rows per application, reference tables for Lebanon's governorate, caza and area hierarchy.
- The workflow states, the code format, the yearly quota per partner and governorate, and the change-decrease justification process are all documented in the guidance note and are in active use by partners. The new system should keep them unchanged so field procedures do not have to be retrained.
- Bilingual English and Arabic UI, with right-to-left support, is a hard requirement.
- Offline-first mobile data collection is a hard requirement. Many field sites have no connectivity.
- The analytics set (dropout reasons, normal versus borderline versus abnormal comparison per age band, partner and governorate analysis, increase scale, uncompleted per NGO, change analysis) defines the reporting scope for the new dashboard.

5. Recommended target architecture on Azure

Kept brief, to be detailed in the next step.

Layer	Recommendation
Containers	One image per service: API, web front end, background worker. Built from this repo with multi-stage Dockerfiles, pushed to Azure Container Registry.

Layer	Recommendation
Hosting	Azure Container Apps for the API and web front end (scale to zero on staging, managed TLS, revisions for blue-green). App Service for Containers is the simpler fallback.
Database	Azure Database for MySQL Flexible Server to keep the data migration a straight dump and restore. Postgres is an option only if we accept a schema translation step.
Files	Azure Blob Storage for partner logos, exports and APK distribution.
Secrets	Azure Key Vault, injected into Container Apps. No secrets in images or repo.
Identity	Keep local accounts for partners and collectors. Add Microsoft Entra ID sign-in for UNICEF admin users.
Observability	Application Insights and Log Analytics.
Delivery	GitHub Actions: build, test, scan, push image, deploy to staging, manual gate to production. Infrastructure as Bicep in the repo.

Application stack options, to decide in the next step:

- **Option A, evolve.** Upgrade to Laravel 11 or 12 on PHP 8.3 and containerise. Lowest risk for data and business rules, fastest path to Azure. UI is modernised with Inertia and Vue or React. Mobile app rebuilt separately.
- **Option B, rebuild.** New API (ASP.NET Core or NestJS) plus a React or Next.js front end, and a cross-platform mobile app (Flutter or React Native) sharing the same API. Cleaner end state, longer timeline, and the scoring and report logic must be re-derived and verified against exported production data.

The recommended sequence either way: export production data first, stand up the current app in Docker locally against that data as a reference environment, then build the new system and verify its scores and reports against the reference before cutover.

6. Proposed step-by-step plan

1. **Secure the assets.** Export the live GoDaddy database and uploaded files. Locate the Android source zip. Rotate the FTP password. (Owner: UNICEF IM focal point.)
2. **Reference environment.** Containerise the existing Laravel 5.8 app as-is (PHP 7.4 image, MySQL 5.7) with Docker Compose, load the production export, and confirm it runs. This becomes the oracle for verifying the new system.
3. **Decide the stack** (Option A or B above) and confirm the Metronic licence position.
4. **Define the API contract and sync protocol** for the mobile app, moving code generation and conflict handling server-side.
5. **Build** the new web and API, with the scoring rules and reports covered by automated tests against data from the reference environment.
6. **Build** the new mobile app against the same API, offline-first.
7. **Provision Azure** with Bicep and GitHub Actions, deploy to staging, run partner acceptance testing.
8. **Cut over**: final data sync, DNS switch, decommission GoDaddy.

Open questions for the next step: which stack option, whether Entra ID is wanted for admin users, whether iOS is in scope for the new mobile app, and whether historical data must be re-scored under the new implementation or frozen as collected.